Mightling: A Vendor-Free Coding Agent on a Single Unified-Memory Workstation, Built on the Codex CLI

dgxcoder
dgxcoder@dreamference.ai

October 2026

Abstract

Terminal coding agents such as the Codex CLI are open source, but they are built around a hosted model, a vendor account and a cloud of companion services. We describe *Mightling*, a coding agent whose model runs on one NVIDIA GB10 workstation (128 GB of CPU–GPU unified memory): a 27B-parameter model (Qwen3.8-27B, NVFP4) served locally through SGLang with a block-diffusion drafter. Source code, prompts and inference never leave the machine, and no vendor account is involved. Mightling is a fork of the Codex CLI (release `rust-v0.158.0`, 4,894 Rust source files, 1.93 million lines). Its central engineering choice is to keep the fork almost identical to upstream. The upstream tree is never edited. Each build exports it, applies a series of 23 patches totalling 43.0 KB (43 files, $+170/-53$ lines), and links in a separate crate that does the actual work of localisation: model discovery, configuration, prompt rebranding, help-text rewriting, a signed release-based updater and refusal of cloud-bound commands. We report how the series shrank from an initial 406 KB to about 9 KB, which techniques did it, and how seven later features each cost only a hook. We also report an inventory of the cloud dependencies we had to switch off, a trace-based audit that keeps them off, an enforced network switch, integration findings that are not visible from the documentation, and the host-safety layer that unified memory forced on us after model loads froze the machine. On a 100-task sample of SWE-bench Verified, run inside each task's container on this machine, the agent resolves 68; calibrated against the 175 published leaderboard submissions on the same tasks, that corresponds to about 69% on all of Verified (95% band 64–73%). A two-session "study, then fix" mode that led by three tasks on a 24-task sample resolved 69 of the 100 at 2.2 times the time, and four prompt and harness changes measured as paired A/B nights on fresh tasks showed no measurable difference; three runs of one configuration on the same 44 tasks resolved 29, 29 and 31, and some run resolved 36. We then report what was built on the fork, each from the same launcher crate or beside it: a terse-answer mode that halves answers to questions and leaves coding work unchanged, a compaction limit tied to the server's KV pool with a rule-built ledger of what summaries lose, a two-layer code index (a tree-sitter knowledge graph plus compiler-exact SCIP data), an overnight task queue that works in git worktrees, and a client/node split that pairs workstations over SSH. We are explicit about what is estimated rather than measured.

1 Introduction

Agentic coding tools now read repositories, run shell commands and edit files autonomously [22–24]. The most capable of them are distributed as open-source clients of proprietary services. The client is inspectable, but a session sends source code, prompts and tool output to a hosted model. Many organisations cannot accept that. Examples are regulated industries, security research, and anyone whose code is itself the asset.

Two developments make a local alternative plausible. First, open-weight models in the 100B+ parameter class, quantised to four bits [6] and accelerated with speculative decoding [4, 14], now generate at interactive speed on a single desk-side machine. Second, the leading agent clients are open source, so a local deployment can reuse years of agent engineering rather than rebuild it. Examples of that engineering are the tool-calling loop, sandboxing, approval flows and the terminal user interface.

Reusing a fast-moving upstream has a cost. A fork that edits the code freely soon stops being able to take upstream releases. A fork that edits nothing cannot remove the parts that assume a cloud. This paper reports how we navigated that tension in Mightling, a local AI stack for the NVIDIA GB10 made by Dreamference; its subject is Mightling's terminal coding agent.

What "local" means here. The model, the agent and all session state run on one workstation. Source code, prompts, tool output and model traffic never leave it, no vendor account exists, and a traced session shows no connection to any vendor (section 5.1). Mightling is not air-gapped, however. Network access happens only through tools the agent or user invokes explicitly:

- web search goes through a SearXNG [17] instance on the machine, which queries upstream search engines on the agent's behalf;
- reading mail uses Gmail over IMAP, and Drive and Calendar their own read-only APIs;
- `ling update` contacts GitHub;
- building `ling` downloads dependencies;
- the phone messengers (Signal, Matrix over Tailscale, Telegram) reach their networks, but each is off until a command turns it on, and the egress audit names every one that is on.

Gmail access can be switched off (`ling_gmail = false`), and so can every network access of the agent: `/airgapped on` gives each command the agent runs an empty network namespace (section 5.1). Updates and builds happen only when the user starts them. No model traffic leaves the machine, with one exception: at the default level, search queries the agent composes go to upstream engines through SearXNG, and they can contain fragments of code or context.

Contributions.

- A **patch-minimal fork method** for a large Rust agent (section 4). The method has four parts: an unmodified pinned submodule, a build-time export, a small series of one- and two-line hooks, and an external crate linked into the binary. We quantify the result: 43.0 KB of patches against a 1.93M-line upstream, of which the localisation itself took about 9 KB. We also report the techniques that reduced the series from 406 KB.
- An **inventory of the cloud surface** of a modern coding-agent client, and how each part was refused, hidden, replaced or repurposed (section 5), with a trace-based audit that re-checks it after every build and a network switch enforced in the command sandbox.
- **Integration findings** for running a vendor agent against a local model (section 6). They cover discovering the served model, the undocumented catalogue schema, a configuration-scoping failure, why Model Context Protocol tools and Code Mode's `exec` were invisible to the local model until they were sent as plain functions, and a compaction limit the server's memory, not the model's window, has to set.

- A **host-safety layer for unified memory** (section 7). It combines pre-load admission checks with a pressure-stall watchdog, and was motivated by real incidents in which model loads froze the workstation.
- A **measured cost model for a local agent's output** and the terse-answer mode built on it (section 6): on this machine one token of prose costs the time of about 67 prompt tokens, and three quarters of the agent's output is tool-call arguments that no speaking style touches.
- A **task-level evaluation on SWE-bench Verified** run on an arm64 desk machine (section 9), with an estimate of the matching overall score calibrated on every published leaderboard submission's per-task results, a classification of every failure, and paired A/B nights that measure how large a change one night can detect.
- An **evaluation** of what exists (section 8), an implemented code index that combines a tree-sitter graph with SCIP and reports which of the two answered (section 10), an overnight task queue (section 11), pairing of several workstations (section 12), and lessons for fork-based agent products (section 13).

2 Background

The Codex CLI. The Codex CLI [7] is an open-source Rust workspace of about 160 crates. It includes a terminal UI, a non-interactive `exec` mode, session resume and fork, a platform sandbox (Bubblewrap on Linux) and an approval model for tool calls. It also has *Code Mode*, in which the model is given a single `exec` tool and writes short JavaScript programs. A separate V8-based host process (`codex-code-mode-host`) runs these programs, and within them every other tool, including Model Context Protocol (MCP) [2] tools, is callable as a function. Codex can talk to "open-source" providers (`--oss`) through a standard chat-completions endpoint. Many product surfaces, however, assume an account with the vendor's hosted service: login, cloud tasks, connectors, usage limits and feedback upload. The self-updater needs no account, but it installs from the vendor's distribution channels.

The machine. The NVIDIA GB10 pairs an Arm CPU with a Blackwell GPU (compute capability 12.1) over 128 GB of shared LPDDR5X. No memory is reserved for the GPU. Model weights, KV cache, the desktop and every other process draw from the same pool. This one fact shapes much of section 7.

The model. Mightling's default model is Qwen3.8-27B in an NVFP4 checkpoint, served by SGLang and accelerated by a calibrated NVFP4 DFlash2 block-diffusion drafter [4, 27] that proposes twelve tokens per step. SGLang serves it because the drafter runs nowhere else yet. The model runs with a 262k-token context and leaves about 38 GB of memory free while serving. Single-stream, it decodes 32.0, 68.6 and 84.1 tokens/s on prose, code and JSON (measured with recorded prompts; an earlier measurement at sixteen draft tokens, whose prompts were not recorded, gave 25.5, 50.3 and 87.0). Agent sessions, where prompts are long and output is mostly tool calls, were measured by replaying recorded sessions against the server: single-stream it decodes about 48 tokens/s at a step of about 103 ms, with about 5.2 tokens accepted per step. Twelve draft tokens replaced sixteen on 2026-10-09 because they measured 7% faster on that replay (48.3 against 45.1 tokens/s; 5.23 against 4.99 accepted per step); eight, ten and fourteen gained nothing. The about 30 tokens/s that production had logged were two concurrent sessions on a loaded machine; two replayed streams on

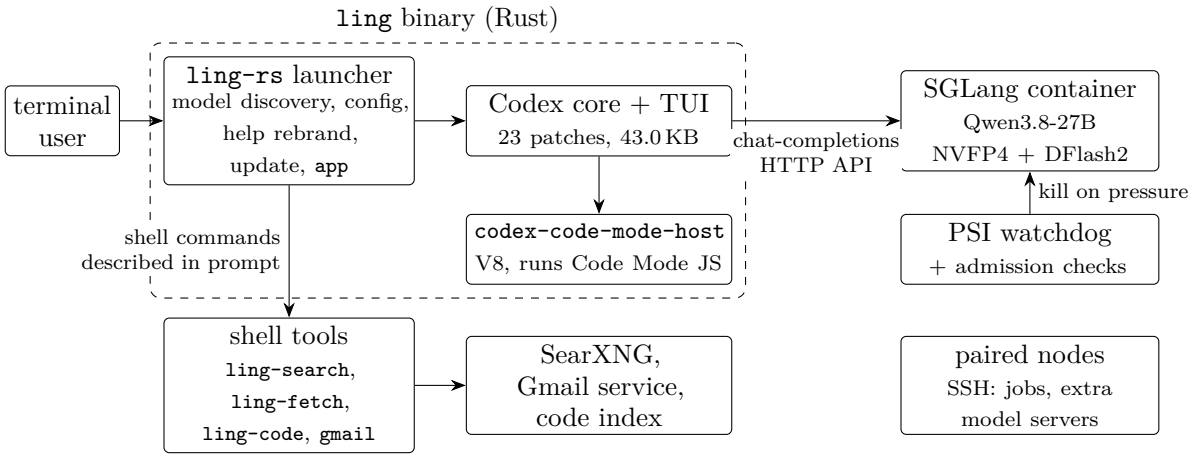


Figure 1: Mightling at run time. The only Codex code paths that differ from upstream are the hooks applied by the patch series; everything Mightling-specific lives in the linked `ling-rs` crate or in separate local services.

an idle machine decode 33.9 tokens/s each. It is the only model served. The default until 2026-09-29 was Qwen3.5-122B-A10B, a mixture-of-experts model with about 10B parameters active per token, in Intel's AutoRound int4 checkpoint [6, 11] with FP8 dense layers, served by vLLM [13] with a twelve-token DFlash drafter [26]. It ran with a 32k-token context, a limit forced by memory headroom (section 7), and decoded 23.8, 49.9 and 53.1 tokens/s on the same three kinds of output; it was kept as a fallback until 2026-10-07 and then removed with its images. The vLLM launcher remains, so the method does not depend on one engine.

3 System overview

Figure 1 shows the running system. The user types `ling`, a single Rust binary. Before Codex parses its command line, the linked launcher crate runs several steps. It finds the local model server and waits for it if it is still loading. It asks the server which model it serves and how long that model's context is. It writes the model catalogue and configuration Codex needs. Finally it prepends the options that point Codex at the local provider. From then on the process is Codex, and every upstream flag and subcommand behaves as documented, apart from the cloud surfaces listed in section 5.

Web search and page fetching are two small Rust programs, `ling-search` and `ling-fetch`, installed beside `ling`; code navigation is a third, `ling-code` (section 10); Gmail reading is provided by `ling-admin`, the Python management CLI. The agent invokes them as shell commands (section 6). The same `ling-admin` manages the model-server container, a browser chat UI built on Onyx [15], the nightly run of queued tasks (section 11), the pairing of other workstations (section 12), SWE-bench runs (section 9), and the build of `ling` itself. A desktop app (Electron, with `ling` bundled) and prebuilt clients for macOS and Windows reach the same model server over the local network. The launcher also serves a browser interface of its own, `ling web`, through which a phone reaches the agent: a Signal bridge reads only the owner's "Note to Self" conversation, and a Matrix or Telegram bridge serves a private chat. Each is off until set up, and the Signal bridge acts on no message while `/airgapped` is `on`. A local file index, `ling-docs`, gives the agent search over the user's documents without opening a socket.

4 A patch-minimal fork

4.1 Build pipeline

Codex is vendored as a git submodule of our own GitHub fork, pinned to the stable release tag `rust-v0.158.0`. The submodule is *never modified*: the builder's tests assert that `git status` inside it is empty after a build. A build proceeds in six steps.

1. Export the pinned commit's `codex-rs/` directory with `git archive` into a scratch tree.
2. Copy the launcher crate, which is ordinary source in our repository, into that tree as `codex-rs/ling`.
3. Apply the patch series with `git apply --check` and then `git apply`, so a patch that no longer fits fails before anything is half-applied.
4. Build `ling` and `codex-code-mode-host` with Cargo, into a persistent target directory. A persistent target means an edit recompiles only the crates it touches.
5. Install both binaries side by side, because Codex locates the Code Mode host next to its own executable. Symlink `~/.local/bin/ling` to the installed binary.
6. Record a *build key*: a hash of the submodule commit, every patch's bytes, the launcher sources and the compiler-profile overrides. An unchanged key skips the build.

The three shell tools are not part of that build. `ling-search` and `ling-fetch` share one crate and `ling-code` has another, each with its own lockfile, built `--locked` and keyed separately, so changing them never relinks Codex.

Moving to a new upstream release means bumping the submodule and resolving whichever hunks stop applying. Because the hunks are few and small, conflicts are local and obvious.

4.2 From 406 KB to 9 KB, then features at a hook each

The first working series had four patches totalling 406,116 bytes. Table 1 shows the current one. Four techniques brought it to about 9 KB.

Rename at run time, not in the tree. 395,156 bytes of the original series came from a single patch renaming the agent in Codex's bundled model catalogue (`models.json`). That file stores each system-prompt template, about 20 KB, as one JSON line, so a one-word change produces a whole-line diff. The launcher now reads the bundled catalogue through the upstream crate's public function (`bundled_models_response`). It takes the longest template, replaces the identity sentence, which also names a model Mightling does not run, and hands the result to Codex in the model catalogue it writes anyway. The patch disappeared.

Rewrite presentation where it is assembled. Help text is spread across doc comments in dozens of files. Patching each string would grow with every upstream release. Instead, one hook routes Codex's argument parsing through a function in the launcher crate. That function walks the `clap` command tree once, rewrites the product name in every `about`, `help` and `bin_name`, and then parses. It leaves paths and identifiers that merely contain the name, such as `~/.codex`, `$CODEX_HOME` and `codex-code-mode-host`, unchanged, because renaming them would describe files that do not exist. Only literal usage strings that clap does not expose remain in the patch.

Use the build system's own unification. Upstream vendors OpenSSL only for musl targets, so a glibc build needed system headers, and we had patched `codex-core`'s manifest to change that. Cargo unifies a dependency's features across the whole build graph. A single `openssl-sys = {features = ["vendored"]}` line in *our* crate's manifest therefore applies to every crate that links OpenSSL, and that patch disappeared too. Similarly, a path dependency located inside the workspace root makes the launcher a workspace member without editing the workspace manifest, and the binary is renamed at install time rather than in Cargo metadata.

One line per behaviour. What remains is mostly one-line hooks, as table 1 shows. Examples: return `false` from a slash command's visibility predicate, mark a subcommand `hide = true`, or send a subcommand's dispatch into the launcher. The largest remaining hunks are presentation strings (patch 0001) and `/usage` (0011), which repurposes a hosted-plan usage screen as local token statistics.

New features cost a hook, not a diff. The series reached its smallest size, about 9 KB, when the four techniques above were done; everything since has been hooks. Up to 0016 they are removals. From 0017 on they are features, and each puts its logic in a crate of ours and only its entry point in Codex:

- `/cavemode` (0017, section 6) and `/night` (0018, section 11) each add a slash-command variant, its description and the dispatch arms that call into the launcher crate (566 and 769 lines there with their tests);
- `/airgapped` (0019, 0023, 0024; section 5.1) is one call in each place a command's permissions are resolved: Linux's sandbox helper, the app server's configuration and Windows' sandbox, all asking one std-only resolver crate;
- flattening MCP tools (0020, section 6) and observation masking (0021, off by default) each route one step of request assembly through a crate of ours; when Code Mode's `exec` had to reach the model as a function too, 0020 grew by 217 bytes, one argument and one line that maps each answer back as it arrives, and the cap was not raised;
- `/apps` without a vendor sign-in (0022) answers Codex's connector list from the launcher, and `/node` (0025, section 12) suspends the TUI to run node management in the real terminal, so a password typed there never enters the session.

A test caps the series' total size, now at 43,000 bytes. The cap is raised only explicitly, in the commit that needs it and with the reason recorded, so the series cannot grow by drift.

4.3 Build-reproducibility details

Three upstream build assumptions did not hold outside upstream's release pipeline.

- **V8 flavour.** Code Mode links V8 built with pointer compression and the V8 sandbox. The `v8` crate's build script downloads a matching prebuilt library [8], but that flavour is not published for `aarch64-unknown-linux-gnu`, so the download fails. Upstream publishes it on its own release page, and the submodule carries a manifest of those files' SHA-256 digests. Our builder fetches the archive and its bindings, verifies each against the committed manifest, and passes them to Cargo, which mirrors upstream's CI.

Patch	Files	+/−	Purpose
0001 brand-mightling-name	12	20/19	Product name in TUI header, status card, slash-command list, `exec` banner and reply label, fixed usage strings
0002 ling-launcher-hook	2	2/1	One dependency line; route argv through the launcher
0005 hide-vendor-login	2	4/0	Hide `login`/`logout` and `/logout`
0006 hide-cloud	1	2/1	Hide `cloud` (kept for a private cloud)
0007 hide-remote-control	1	2/0	Hide `remote-control`
0008 mightling-update	1	2/2	Send `update` to the launcher's release updater
0009 remove-feedback	2	3/1	Remove `/feedback` (uploads session logs)
0010 hide-voice	1	4/1	Hide `/voice` (hosted realtime API) and `/pets` (art fetched from the vendor's CDN)
0011 usage-token-stats	4	7/17	`/usage` shows local token statistics
0012 hide-auto-review	1	2/0	Hide `/approve` (defaults to a hosted review model)
0013 disable-usage-analytics	1	2/1	Construct the analytics client disabled
0014 mightling-home	1	2/0	Use `~/.mightling`, never upstream's login
0015 no-vendor-network	4	20/4	Close the channels found by tracing (§5.1); drop Codex's own tips
0016 doctor-offline	2	20/0	Switch off `doctor`'s sign-in, update and reachability checks
0017 cave-mode	5	10/0	`/cavemode`; register the terse-answer section the launcher supplies
0018 night-slash-command	2	7/0	`/night`: queue a task for the overnight run
0019 airgapped	8	17/2	`/airgapped`; enforce `on` in the Linux sandbox helper; Full Access refused at `on`
0020 flat-mcp-tools	3	4/1	Send MCP tools, and Code Mode's `exec` where the server would drop it, as plain functions (section 6)
0021 observation-masking	2	5/1	Optionally replace old tool outputs by a placeholder naming a saved copy
0022 ling-apps	2	6/1	`/apps` lists local Gmail, Drive and Calendar connectors, no vendor sign-in
0023 airgapped-app-server	2	2/0	The app server refuses Full Access at `on` for every client
0024 airgapped-windows	4	12/0	The same enforcement in the Windows sandbox
0025 node-slash-command	4	15/1	`/node`: provision, pair and check nodes from the TUI (section 12)
Total (23 patches, 42,958 bytes)	43	170/53	against 4,894 Rust files, 1.93M lines

Table 1: The patch series at the time of writing. Numbers 0003 and 0004 were retired when their content moved into the launcher crate. The total counts each upstream file once; the per-patch column counts it once per patch.

- **Lockfile versions.** Upstream's `Cargo.lock` records the 158 workspace crates at version `0.0.0`, which its release job bumps just before building. A `--locked` build therefore fails. The only entries Cargo rewrites are those 158 workspace versions, so we build without `--locked` and have confirmed that every third-party pin is unchanged.
- **Debug information.** The upstream release profile keeps line tables and strips only at packaging. Built as-is, the agent binary was 1.4 GB. Setting `CARGO_PROFILE_RELEASE_DEBUG=none` and `_STRIP=debuginfo` through the environment, which needs no patch, yields 315 MB. This also matters at run time (section 6).

5 Cutting the cloud cord

We audited every subcommand and slash command of Codex 0.158.0 for a dependency on vendor-hosted services. Table 2 lists what we changed. We used four treatments, deliberately distinguished:

- *Refuse*: the launcher rejects the command before Codex parses it, with an explanation.
- *Hide*: the command is removed from help and menus, but its code stays compiled for a future private-cloud implementation.
- *Replace*: the same command does something local.
- *Remove*: the command is invisible and unreachable.

Hiding alone is not a security boundary: a hidden clap subcommand still parses. Four commands are therefore also refused in the launcher: `login` and `logout`, which start or end a vendor sign-in, and `cloud` and `cloud-tasks`, which browse tasks on the vendor's servers. `remote-control`, `/voice` and `/approve` are only hidden. Each needs a vendor account that Mightling does not have, so none of them can succeed. The refusal finds the subcommand the way Codex will: it reads Codex's own clap definitions to learn which options consume the next token, so `ling -c k=v cloud` is refused too. An earlier version checked only the first argument and let that form through.

5.1 Traffic an audit of commands does not find

Auditing commands found every channel a user starts deliberately. It found none of the channels Codex opens by itself. We found those by tracing: a real `ling exec` session, and a real interactive session driven through a pseudo-terminal with a fresh configuration directory, each under `strace -f -e trace=connect,execve`, with the HTTP client's debug logging on to name the URLs. With the model served on the loopback interface, anything else in the trace is traffic we did not intend.

The first traces were not clean, although no vendor login was present. Codex exported OpenTelemetry metrics to Statsig at `ab.chatgpt.com`, the default in release builds, with an API key compiled in; it ran `git ls-remote https://github.com/openai/plugins.git` to sync the vendor's curated plugins; it requested `chatgpt.com/backend-api/plugins/featured`; and the interactive interface fetched an announcement from `raw.githubusercontent.com` on every start. Two further channels needed the login that an earlier upstream installation had left in the shared `~/.codex`: usage analytics and token refresh.

Each is closed where it originates rather than by configuration, so no setting or later login can reopen it. The analytics client and the metrics exporter resolve to "disabled", the plugin sync and the two fetches return before any request, and Mightling keeps its state in `~/.mightling`, copying sessions and settings across but never the login. As a second line, the launcher points Codex's

`chatgpt_base_url` at a closed local port, so any vendor-backend call we did not find fails on the machine. Re-traced, both sessions contacted only two loopback services: the model server and the local mail service. The lesson is general: for a vendor client, the list of what it sends is an empirical question, and has to be re-measured after every upstream update.

The audit as a command. That trace is now `ling-admin audit egress`, and every build of `ling` runs it after installing a new binary. It runs one real `ling exec` session in a throwaway repository and home under `strace -f -e trace=connect,sendto,sendmsg,sendmmsg,execve` and passes only if every destination is an allowlisted loopback port (the model server, the mail service, SearXNG), no DNS query was sent and no git command reached a network. A trace that itself failed is never a pass. Two details came from recordings: glibc sends name lookups with `sendmmsg`, so tracing `connect` alone misses a query, and a connection to the local stub resolver at `127.0.0.53:53` is loopback yet still a query that leaves the machine. A phone messenger's bridge talks to its network by design and outside the agent's process, so the trace cannot see it; instead every report names each bridge that is switched on as a declared exception, and none is ever passed over in silence.

An enforced network switch. `/airgapped` has two levels. At `off`, the default, the agent's commands may use the network. At `on`, every command the agent runs gets an empty network namespace. The hook is in Codex's sandbox helper, not in its core: core derives a command's network policy in about a dozen places, while every sandboxed command passes through the helper, which runs in the command's own environment and so finds the session's level by its thread identifier. The level resolves from the session, then an environment variable, then the stricter of the repository's and the user's configuration, because the agent can write the former. Full Access has no sandbox, so the two are never allowed together: the launcher, the permission menus and the app server each refuse the combination and say why. A third level, search through one engine only, was built and then removed when that engine answered SearXNG with a CAPTCHA, so the level searched nothing.

Updates. `ling update` replaces Codex's installer-detection logic. It queries the latest *published* release of Mightling's repository, compares it with the version stamped into the binary at release build time, and downloads the gzipped `ling` and `codex-code-mode-host` with a SHA-256 manifest. It verifies both files before replacing either, and swaps them in next to the running executable, so a failed download leaves the installation intact. A shell installer downloads the same assets, with the same checks, on a fresh machine. Releases with prebuilt binaries have been published since 1.3.0 (the first one lacked them and was withdrawn, section 13). Releases from 1.5.1 on, the first under the new name, are *signed*: the checksum file lists every asset and carries an Ed25519 signature made in a protected CI environment, the public key is compiled into `ling`, and both the updater and the installer refuse a release from that version on whose signature is missing or does not verify. 1.5.1 and 1.6.0 were published this way. The environment first required a person's approval for each signature; since 2026-10-12 a release run publishes itself, its notes taken from the repository, and the key stays usable only from the main and release branches. An upgrade from a real 1.4.1 installation, including the rename's one-time migration of the home folder, was exercised in a scratch home.

Surface	Why it does not fit a vendor-free agent	Treatment
`login`, `logout`, `/logout`	Vendor-account / API-key sign-in; no account exists	refuse + hide
`cloud`, `cloud-tasks`	Browses tasks on the vendor's servers	refuse + hide
`remote-control`	Relays sessions through the vendor's servers	hide
`update`	Installs upstream Codex over Mightling	replace (verified release download)
`/feedback`	Uploads session logs to the vendor	remove
`/voice`	Hosted realtime voice API	hide
`/usage`	Hosted-plan usage and limit resets	replace (local token statistics)
`/approve` (auto-review)	Defaults to a hosted review model	hide
`app`	Opens the vendor's closed-source desktop app	replace (opens Mightling's own window)
update check at start-up	Compares against upstream Codex releases	disabled in config
Found only by tracing a session (§5.1)		
usage analytics	Posted to `chatgpt.com` whenever a vendor login is present	disabled
shared `~/.codex`	Inherited an upstream vendor login and its token refresh	own `~/.mightling`
OTEL metrics	Exported to Statsig (`ab.chatgpt.com`) by default in release builds	disabled
curated plugin sync	`git ls-remote` of the vendor's plugin repository on GitHub at start-up	removed
featured plugins	Request for featured plugins to `chatgpt.com`	removed
announcement tip	TUI fetch from GitHub's content host on every start	removed

Table 2: Cloud-coupled surfaces and their treatment. On Linux, Codex's `app` subcommand is not compiled at all, so Mightling's `app` is new.

6 Running a vendor agent against a local model

Discover, do not configure. The local server's model can change between sessions, and an earlier version of our launcher left Codex advertising the previous model with the wrong context length. The server's `/v1/models` endpoint reports both the served model's identifier and its `max_model_len`. The launcher therefore derives the model catalogue entry from the running server, with no copy of the model registry: context window, auto-compaction limit and truncation policy all come from that value.

That value turned out not to be the whole truth. The default model is launched with a 262,144-token context and reports it, but the server's KV-cache pool, sized at launch from free memory and shared by every stream, held 156,907 tokens when we looked. Both the window the catalogue advertises and the compaction limit derived from it were therefore above what the server can hold. None of the 111 interactive sessions recorded at the time contained a compaction (the largest peaked at 71,421 prompt tokens), but a deliberately extreme limit of 6,000, set through Codex's own `model_auto_compact_token_limit`, showed what a compaction costs here: one small task compacted seven times and ran thirteen commands where three were needed, because the model re-ran what each summary had dropped. Two changes followed, neither a patch.

- *The limit follows the pool.* At every start the launcher reads the pool from the server's metrics and passes 60% of it as the compaction limit unless the user set one: 94,144 tokens on the default model, against the 262,144 the catalogue advertises.
- *A ledger replaces what summaries lose.* Codex's summary keeps the conversation's prose but drops every tool call and output, so which files were touched and which commands failed survives only if the summary happens to name it. A hook that Codex runs after each compaction calls the launcher, which rebuilds by rule, with no model, in about 15 ms, the files the session touched, the commands that failed since the last compaction and the last test result, and hands them back as context. On one long task at a 32k limit, the session with the ledger compacted 11 times instead of 24 and ran 114 commands instead of 245. It has been on by default since 2026-10-02; registering it as a hook Codex trusts meant recomputing Codex's own hash of the hook's definition, which a test pins.

Benchmark sessions compact far more than interactive ones, since each task runs at a limit of 49,152 tokens. Across seven benchmark runs of 320 tasks in all, tasks that compacted at most once resolved 75–78%, twice 57%, and three times or more 38%; compaction took 17% of all agent time, a median of 98 s each with two sessions sharing the server. Part of that gradient is that hard tasks run longer, but on the 18 tasks that ran both ways the low-compaction runs still resolved more (50% against 40%, too few to measure a size). After a compaction only 1% of commands repeated an earlier one exactly, so the ledger does what it was built for. A third change, compacting at the end of a turn while the user reads the answer instead of in the middle of the next one, is built as a setting and off until a benchmark of interactive sessions decides it. We also tried a 0.5B diffusion model, run as a sidecar, as a compressor of tool output: of 50 file names, hashes and numbers in a `git log`, 14 survived, so it has no role in compaction. It was unusable in every other role we tried as well, and the sidecar has been switched off since 2026-10-03.

The catalogue is a hard schema. Codex refuses a model it has no catalogue entry for, and parses entries with strict `serde` structs. A wrongly shaped field is a start-up failure, not an ignored key. Several fields have non-obvious shapes:

- reasoning levels are `{effort, description}` records;

- `visibility` is one of `list|hide|none`;
- truncation is `{mode, limit}`;
- `base_instructions` is required in practice, and it is the *entire* system prompt, not a label.

An early version filled `base_instructions` with a one-line placeholder. The model then had no description of its tools, guessed tool names that do not exist, and fell back to `curl` for work a tool was provided for. Supplying Codex's own template, with the identity sentence rebranded, fixed this.

Configuration scoping. Codex's `config.toml` accumulates tables written by Codex itself, for example `[tui.model_availability_nux]`. Appending a top-level key such as `model_catalog_json = "..."` to such a file silently makes it a member of the last table. The result is the start-up error *invalid type: string, expected u32*. The same failure recurred later with a different table. The launcher now edits the file as a syntax tree (`toml_edit`). It sets keys that must follow the server unconditionally, and adds keys that the user may override only when they are absent.

Tools as shell commands, and why MCP failed. Code Mode exposes MCP tools only inside its JavaScript runtime, as `tools.mcp__server__tool(args)`. We wired a SearXNG [17] MCP server for web search. The server initialised and its tools appeared to reach the model's context, but the local model repeatedly called the namespace directly, received *unsupported call*, and gave up. Plain shell commands described in the system prompt worked reliably: `ling-search`, `ling-fetch` and, later, `gmail search|read|status`. We first read this as a property of the model. It was not: Codex sends MCP tools in a `namespace` tool type of its own, and both SGLang and vLLM drop tools of a type they do not know, so no MCP server, the code index's included, had ever been visible to the model. Patch 0020 sends each MCP tool as a plain function, so it now reaches the model; the shell commands remain the primary path. The same filter hid Code Mode itself. Codex offers Code Mode's `exec`, and the free-form variant of its `apply_patch`, as *custom* tools whose argument is raw text under a grammar, and the servers drop that type too: in eleven recorded turns the model never called `exec`, because it never saw it. On a server other than our own engine the launcher's tool crate now offers `exec` as a function with one string parameter, the grammar appended to its description, turns the model's function call back into the custom-tool call Codex expects before Codex records it, and replays earlier calls in the form the model saw. The free-form `apply_patch` is offered only to our own engine, which accepts custom tools; elsewhere the agent edits files through the shell as before. Both were then measured as paired A/B nights (section 9). With `exec` offered as a function the model called it in every one of 44 benchmark tasks, 2,865 times, so the translation works; the `apply_patch`-as-function arm never started, because the pinned Codex's catalogue parser accepts only the free-form variant. The lesson is about where to look: a local serving stack can silently filter what a vendor harness sends, and the request that reaches the model, not the one the harness built, is the thing to inspect.

Timers sized for a hosted service. Before the first token, the only timer Codex runs is a 300 s stream-idle timeout, and it covers the server's queue as well as prompt processing. One stream fits: a prompt that fills the whole KV pool takes about three minutes. Several queued streams do not, and an overnight run hit the limit. The launcher now writes a 900 s timeout into its provider entry when the user has not set one. Two smaller defences came from the same review: `server start` checks that a tool call survives the streamed path `ling` uses and names the parser or chat template likely at fault if it does not, and every process that reaches the model exempts loopback and the model server from any proxy the environment leaves behind.

Prose is the expensive output. Hosted agents are priced per token, so the usual argument for terse output is money. Locally the currency is time, and its exchange rate is lopsided. This model decodes prose at 25.5 tokens/s and reads a prompt at about 1,700, so one token of prose costs the time of about 67 prompt tokens; and 81% of the input of our recorded sessions came from the prefix cache. A few hundred tokens of standing instructions are therefore nearly free, while every sentence of answer is not. We also measured where the output goes: across 52 recorded sessions with tool calls, 75.5% of output tokens were tool-call arguments, 23.1% final answers and 1.3% commentary between calls.

Mightling therefore answers tersely by default. A *cave mode* level (`off`, `lite`, `full` or `ultra`; `/cavemode` switches it mid-session) is delivered as a section of model-visible session state that the launcher crate contributes through Codex's extension registry: the level's text once, then a one-line reminder at the start of each later user turn. The system prompt is untouched, so `off` is exactly upstream behaviour. In a benchmark of 532 single-turn runs and 11 sessions of eight or nine turns, the default level, `ultra` (at most three sentences of prose outside code unless the user asks for more), cut answers to questions to 0.42–0.57 of their length at `off` and wall time to 0.41–0.74, and passed 35 of 36 correctness checks, the same number as `off`. On coding tasks no level moved answer length or wall time beyond the noise. This reproduces locally what a paired study of the public "caveman" skill found on a hosted agent, about 8.5% fewer output tokens with quality unchanged [18], and for the same reason: an agent's output is mostly tool calls and code. Three further observations shaped the design:

- *Content rules work; grammar rules do not.* Answer-first, one option instead of a menu, no unrequested code and a numeric sentence cap shortened answers. Dropping articles did not happen (the article rate stayed between 5.9% and 9.5%), and would have saved little: in this model's tokenizer the usual abbreviations are one token, as are the words they replace.
- *Numbers hold; adjectives do not.* "At most three sentences" is followed; "short" barely is.
- *A level fades over a session unless it is repeated.* Without the per-turn reminder the cap stopped holding after a few turns; with it, turns five to eight of a session summed to 0.49 of the same turns at `off`.

We compress only what the model writes. Compressing what it *reads* is reported to make models answer longer and less accurately [1].

Untrusted content. Email gives an agent that holds a shell a prompt-injection channel. Gmail access is read-only by construction. `gmail read` frames every message between *BEGIN EMAIL (untrusted)* and *END EMAIL* markers, and the prompt section is added only when an account is connected, and states that email content is never instructions. We regard these as mitigations, not a defence.

Binary size is prompt latency. The pre-rewrite launcher recovered Codex's prompt by scanning the installed binary for JSON-escaped templates. At 1.4 GB this took roughly 0.8 s per launch, and about 0.2 s after stripping, as measured during development. Reading the template through the crate's API removed the scan entirely.

7 Host safety on unified memory

With no dedicated GPU memory, a model load competes with the desktop for every page. We learned this the hard way. On 14 August 2026, six loads in one hour took the workstation down hard

enough to need the power button. None of the freezes invoked the kernel OOM killer. Driver-pinned pages are unreclaimable, and they are not charged to any process the killer would select, so the kernel spun in direct reclaim instead of killing anything. Mightling's server manager now enforces two layers.

Admission checks (before a load). All conditions are checked and reported together, because each fix needs its own `sudo`.

- *An OOM handler* (systemd-oomd or earlyoom) acts on pressure-stall information [20] before the livelock forms.
- *At least 64 GB of swap* gives the kernel somewhere to put cold anonymous pages.
- `vm.min_free_kbytes` *at 1 GiB or more.* The stock value is about 44 MB on this machine, which a loader reading gigabytes per second exhausts between reclaim passes.
- `vm.watermark_scale_factor` *at 200 or more*, so background reclaim starts earlier.
- *sysstat installed*, because its history is the only pressure record that survives a hard reset.

The container is also nominated as the preferred OOM victim (`oom_score_adj` = 800). `ling-admin host setup` applies what the checks report, and the installer runs it on a GB10, together with an AppArmor profile that lets Bubblewrap create the sandbox's user namespace (Ubuntu refuses that to unconfined processes, and a systemd unit, unlike a terminal under an IDE, is unconfined). Installing a second GB10 found one gap: killed partway through a swap resize, the step left the machine with no swap at all. It now completes an interrupted resize when run again.

Watchdog (during a load). A thread samples `/proc/pressure/memory` once per second. It resolves the container's cgroup and kills the container on either of two rules:

- the "full" stall share over 10 s stays at or above 60% for 5 s, matching systemd-oomd's default limit;
- the 60 s average reaches 25%.

The second rule catches pressure that oscillates around the first threshold, resets its timer on every dip, and grinds indefinitely. The kill has three paths, tried in the order the host can still afford:

1. a direct `SIGKILL` to the cgroup's processes, which needs privileges the tool normally lacks;
2. a single request over Docker's Unix socket, with no fork;
3. the Docker CLI, last, because forking a Go binary is the work least likely to be scheduled during a reclaim livelock.

Consequences for serving. These margins are why the earlier 122B model ran at a 32k context rather than 131k: a longer context was refused at start-up for want of 3.3 GiB of KV cache. (The 27B model leaves about 38 GB free and runs at 262k.) They also fixed the order of start-up while a diffusion sidecar existed: it started *before* the model server, with a fixed 8 GB cgroup cap, because the server's pre-flight reads current free memory. A resident sidecar is accounted for, whereas the reverse order lets a marginal check pass and then lose the memory mid-load. Every heavy background job we have added since runs under `systemd-run -p MemoryMax=... -p MemorySwapMax=0`, so

Metric	Value
Upstream (Codex `rust-v0.158.0`)	4,894 Rust files, 1.93M lines
Patch series, initial → smallest → current	406,116 B (4) → ≈9 KB → 42,958 B (23 patches)
Upstream files touched / lines changed	43 files / +170, −53
Launcher crates (`ling-rs` and its sub-crates)	35,900 lines, 553 tests
Code-index router (`ling-code`)	10,444 lines, 131 tests
Agent binary, unstripped → stripped	1.4 GB → 315 MB (+ 93 MB Code Mode host)
Incremental rebuild after a launcher edit	≈7 min (thin LTO relink)
Management and services (Python)	155 modules, 45.5k lines; 1,269 tests
Live slash-command suite, Qwen3.8-27B†	77 tests: 75 pass, 2 skip; 651.3 s
Live slash-command suite, Qwen3.5-122B†	77 tests: 75 pass, 2 skip; 806.7 s
Decode throughput, single stream, 262k context	prose 32.0 / code 68.6 / JSON 84.1 tok/s
Decode, replayed agent sessions, 1 / 2 streams	48.3 / 33.9 tok/s each; 5.2 of 12 accepted
SWE-bench Verified, 100-task sample (section 9)	68 resolved; ≈69% overall (64–73%)
Code-index query, warm, this repository (20 runs)	p50 0.09 s, p95 0.11–0.12 s
Overnight task, live (one task, section 11)	7 s; test passed; checkout untouched

Table 3: Measurements on one GB10 (128 GB). Sizes are binary megabytes. Single-stream throughput was measured on 2026-10-09 at twelve draft tokens; the replay rows on the second GB10 with production's image and flags (two streams at sixteen draft tokens). †Run with the 16-patch build of section 4's first version, before the 122B model was removed; not yet re-run on the current series.

exhaustion kills the job instead of the host. Index runs share one slice and one memory ledger (section 10); each overnight task has a scope of its own (section 11); and while a night run holds its lock, a model load, a build of `ling` and a re-index all refuse to start.

8 Evaluation

We evaluate what the system does today. Task-level coding accuracy, on SWE-bench Verified [12], is the subject of section 9.

Live slash-command suite. A test drives every slash command of the pinned Codex source on a pseudo-terminal, in a throwaway git repository and `CODEX_HOME`, against the running local model. It asserts per command that the expected screen or reply appears, and that removed commands are rejected. Table 3 gives the totals. Two commands that upstream labels "DO NOT USE" are skipped. An earlier run, before the fixes in this paper, had three failures, each diagnosed from the session transcript.

1. *Fresh-home start*: with an empty configuration directory, `ling` opened on Codex's vendor sign-in screen. The start-up account check reads the configured `model_provider`, not `-oss`; the launcher now also passes `-c model_provider=...`, and the test passes.

2. `/btw`: an alias of `/side`, unavailable until a conversation exists. The test must converse first; this was a harness bug.

3. `/init`: first a harness bug (a single missed "esc to interrupt" marker between tool calls ended the wait early), fixed by requiring the marker to stay away for three seconds. The remaining failures were the model: it explored the repository, announced "I'll create a concise AGENTS.md", and ended its turn without the tool call. The same prompt through `ling`

`exec` wrote the file in 6 of 6 runs, with or without an extra "act, don't announce" instruction, so we did not change the prompt. The test now does what a user would: it says "go ahead" once, and fails only if the file still does not appear.

Throughput. On both models we served, decode speed for code is about twice that for prose (68.6 against 32.0 tokens/s on the 27B model at twelve draft tokens, 49.9 against 23.8 on the earlier 122B), and JSON is faster still (84.1 and 53.1). The reason is speculative decoding: per step the 27B model accepts 3.4 drafted tokens on prose, 4.4 on code and 5.9 on JSON. It suits an agent whose output is dominated by tool calls and code: 75.5% of output tokens in our recorded sessions were tool-call arguments (section 6).

9 Task-level accuracy on SWE-bench Verified

Running it on a desk machine. `ling-admin swe-bench` runs `ling` inside each instance's own container on the GB10 and grades the patches with the official harness. Several things had to be found first.

- *Images.* The official instance images are built for `linux/amd64` only. The only arm64 images are a community build covering 400 of the 500 instances [10]; the harness is pointed at them through a local dataset whose image column is rewritten.
- *Validation.* An image existing proves nothing, so an instance is used only once its gold patch resolves and a no-op patch does not, both checked here.
- *The agent in a foreign userland.* The installed `ling` needs a newer glibc than the images have, so the runner builds a copy relinked against the host's loader and libc and mounts it read-only.
- *The container is the sandbox.* Codex's own sandbox cannot start inside a container, so the agent runs with it bypassed, on an internal Docker network that reaches the model server and nothing else.
- *The model to itself.* A benchmark run shares the model server with whoever else uses it, and an interactive session in the middle of a run would change both. The server now sits behind a small streaming proxy, the *model gate*, on the public port. While a run holds it, a request from outside the run's network is answered with a 503 that names the run and the time left, and the user can let requests through for a while; the run's report names each such pause.

Each instance gets a 45-minute limit, 8 GB of memory and four CPUs; two run at once from the 100-task sample on (the 24-task rounds ran up to three). Nothing is chosen by running tests: one session, one patch, as a single attempt in the leaderboard's sense.

From a local subset to an overall score. A local result on a few dozen tasks is not a leaderboard score, and our sample is not average: across the 175 submissions that publish per-instance results [19], systems resolve on average 5.9 points fewer of our 24 tasks than of all 500. We therefore estimate the overall score a local result corresponds to, with two estimators fitted on those submissions: a regression of overall on subset score, and a one-parameter item-response (Rasch) model fitted on the full 175×500 matrix, in which a local round's ability is the one whose expected count on its tasks equals its resolved count. Back-tested by predicting each published submission's overall score from its 24 subset results alone, the Rasch estimate has an RMSE of 5.3 points (the regression 5.1); the 95% bands below are the Rasch estimate ±1.96 times that error.

Results on 24 tasks.

- *Default configuration.* Six rounds with the default prompt resolved 17, 16, 17, 15, 16 and 15 of the 24 tasks. The median, 16, corresponds to about 71% on all of Verified (Rasch 70.9%, regression 70.3%; band 60–81%). On exactly these 24 tasks, 16 is what several published systems built on much larger hosted or open models scored, with overall scores between 66.6% and 77.6%.
- *Study, then fix.* A *refine* mode runs two sessions: one that may read and run the code but changes nothing writes a structured description of the task (intent, requirements, every code path involved, edge cases, what must not change, acceptance checks), and a fresh session fixes the task with that description, told that the task text wins where they disagree. Its one round resolved 20 of 24 ($p = 0.25$ against the matching default round), at a median of 16 min 22 s per instance against 4 min 7 s. In three instances the study wrote down the wrong thing and the fix followed it.
- *The same weight class.* The best published result on these 24 tasks from a model of 20B to 40B parameters is 15 (with more than one attempt); the best single attempt is 14. Those are 2025 models; no Qwen 3.5, 3.6 or 3.8 model has a published submission.
- *The code index.* In the first paired rounds with and without the index of section 10, both resolved 13; the agent never queried the index, so that pair says nothing about it. The harness now reports when that happens.

Results on 100 tasks. On a 100-task sample, published systems score only 1.0 to 1.5 points below their overall score, and the back-test error falls to 2.4 points, so the band narrows to about ±5. The default configuration resolved 68 of the 100, about 69% on all of Verified (band 64–73%), consistent with the 24-task estimate. Paired with it, refine resolved 69: each mode alone resolved about ten tasks the other missed ($p = 1.0$), at 2.2 times the time per task. The 24-task lead did not hold, and refine stays opt-in.

Why the 32 failed. We read every failed run against the task's difficulty, measured as the share of the 78 published submissions scoring 60% or more that solve it. Nineteen of the 32 failures are tasks fewer than 40% of those systems solve. By category: a second code path or file missed (8), the wrong file (5), the right place but a wrong detail (5), behaviour the issue did not ask for (5), a fix that broke other tests (4), the benchmark's test patch blocked by the agent's own test edits (2), stopped mid-work (2) and out of time (1). None came from an empty patch, a tool or environment error, or a malformed tool call. The easy losses came from how the agent treated tests, not from finding the bug: in four of the seven failed tasks that at least 75% of those systems solve, the source change was in the right place and a test decided the outcome, for example a change that broke an existing test, after which the agent rewrote the test. The reading also found two defects in our harness, both fixed.

What one night can measure. The failure analysis proposed changes to the task prompt and the harness, and the tool findings of section 6 two more. Each was measured as a paired A/B night on tasks outside the 100: two arms on the same 44 or 50 freshly validated tasks, each arm one session per task.

Night	Candidate arm	Tasks	Default	Candidate (p)
1	three sentences of test discipline	50	34	33 (1.0)
2	the same, second sample	44	29	31 (0.73)
3	the issue as a contract to check	44	29	26 (0.51)
4	restart after a second compaction	44	31	28 (0.45)
5	`apply_patch` offered as a function	44	29	void (see below)
6	Code Mode off: `exec` left to the server to drop	44	31	30 (1.0)

None of the five that ran differs measurably from the default, which stays. The default arm's runs of nights 2–4 on the same 44 tasks say why: they resolved 29, 29 and 31, only 23 tasks were resolved every time, and 36 were resolved by at least one of the three. A quarter of a run's resolved set is therefore the luck of that run, and a single night can confirm only a large effect; the arms are judged first by the behaviour they target and only then by the count. The same runs bound what choosing among several attempts could gain: a perfect chooser over the three would resolve 36 of 44, about 14 points above a single run's average, at three times the agent time.

Night 5 is void. Its candidate arm failed at start in all 44 tasks: the launcher wrote `function` as the `apply_patch` tool type into the model catalogue, the pinned Codex's parser accepts only `freeform`, and every session ended within seconds with no tokens; the round still exited 0, and only the report at the end of the round showed 0 of 44 with 44 agent errors. The chain's pre-flight had checked that the binary read the override, not that a session starts. Every night now begins with one real task per arm, with the arm's exact options, and the unit does not start unless each session ran to its end (a wrong patch passes; a dead session does not). The default arm, whose tools were those of night 6's candidate, resolved 29.

Night 6: Code Mode. The default arm ran Code Mode with `exec` offered as a function (section 6) and resolved 31 of 44 at a median of 7 min 7 s per task, with six timeouts; the candidate left `exec` as the custom tool the server drops, so the model saw the shell tools directly, and resolved 30 at 5 min 38 s with one timeout (+2.3 points for Code Mode; McNemar $p = 1.0$). Code Mode stays on. The time is the finding. An offline reading of every session's rollout shows a request costing the same in both arms (5.03 against 5.01 s) and the Code Mode arm making 11% more of them (6,353 against 5,738). Three causes account for most of it. 221 of its 2,864 `exec` cells (7.7%) failed before running anything (Python heredocs inside a JavaScript string, a nested tool called without its `tools.` prefix, tools that are not in the isolate), each a full round trip, and 12% of the cells right after a compaction failed against 6.9% elsewhere. 2,646 of the cells (92%) wrapped exactly one shell command, at 2.80 s per call against 2.43 s for a direct shell call in the same arm. And `exec`'s description adds about 1,000 tokens to the fixed prefix, which cost five more compactions (60 against 55). Of the six timeouts, two trace to failed cells after a compaction, one is unclear, and three would have happened without Code Mode. The reading also corrected the harness: its report sums tokens over completed turns, and a session the timeout kills never completes one, so the printed 111M against 160M input tokens is wrong; from the rollouts the arms used 194.7M with Code Mode against 174.0M without (+12%). No task used what Code Mode adds (11 cells composed two tools). The remedies the reading proposes, telling the model in `exec`'s own description when a direct call is right, restating the convention after a compaction and naming the fault when a cell fails, are each an arm to measure.

These are estimates of what this agent, model and machine would score on all 500 tasks under these conditions, not leaderboard results. They do not account for training-data contamination, the

Language	Indexer	Runs / state on this machine
Python	scip-python	every repository; static once its environment is controlled
TypeScript, JavaScript	scip-typescript	every repository; 732 files in about 2 s, 271 MiB peak
Go	scip-go	on request, offline enforced; 21 MiB peak on a fixture module
Rust	rust-analyzer	trusted repositories only: it runs build scripts and procedural macros
Java, Kotlin	scip-java	trusted only, on a scratch copy; needs a recorded JDK 17+; run on a small Gradle project
C#, VB	scip-dotnet	trusted only, on a scratch copy; needs a recorded .NET SDK 8+; run on a console project
C, C++	scip-clang	no linux-arm64 build upstream; universal layer only

Table 4: The exact layer by language. Indexers that execute project code run only in repositories the user has marked trusted.

quantised model, the community images and our validation filter, or the run-to-run variation that the nights show to be several points.

10 Compiler-exact code navigation

An agent that navigates with `rg` and file reads spends many rounds on the 4,894-file upstream workspace, and it silently misses calls through traits, generics and re-exports. We surveyed open-source code-intelligence tools with the constraints of a vendor-free AGPL product in mind: a compatible licence, no network or telemetry, and arm64 availability. The result is a two-layer index, implemented as `ling-code`, a Rust binary of its own beside `ling`.

- *Universal layer*: codebase-memory-mcp [9, 21]. It is a single MIT-licensed C binary with 158 tree-sitter [3] grammars. It builds an approximate call graph for every language.
- *Exact layer*: SCIP [16] indexes produced by compiler front-ends (table 4), converted to SQLite by the `scip` tool.

`ling-code` answers `def`, `refs`, `callers`, `callees`, `impl`, `impact`, `show`, `outline` and `search` by reading both stores directly as SQLite. It joins them through a definition's location, tags every row *exact* or *heuristic*, and the agent's prompt requires verification when a heuristic row decides a rename, a signature change or a deletion. This follows the graph-guided localisation idea of LocAgent [5] and the multi-view serving of CodeNib [25], specialised to a single self-contained machine.

Why two layers. We measured the universal layer against the exact one before building the router. At file level, its call edges recalled 78% of what rust-analyzer's SCIP index holds for the upstream workspace (15,660 functions with cross-file references), and 58% of what scip-python holds for our own Python package: 86% for functions but 52% for methods. For a method of one class that shares its name with a method of another, it found no caller of either, where scip-python resolved each call to the right class. The approximate layer is good enough to orient and not good enough to rename by.

What measurement changed. Several assumptions of the first design did not survive the tools.

- *No per-query command line.* The universal layer's CLI takes 6.6 s per invocation, all of it fixed start-up, while the same question as a read-only SQL query on its database takes under 10 ms. The router reads the database.
- *Exact Rust indexing is expensive, and now measured.* The first attempts under an 8 GB cap were OOM-killed inside the cap. Under a 22 GiB cap, `rust-analyzer scip` on the upstream workspace completes in 691 s cold and 569 s warm, with a peak of at least 21.3 GiB (cap-bounded, so the unconstrained peak is unknown). There is no cheaper mode: its SCIP path always runs build scripts and procedural macros. Such a run is admitted only when the machine can spare the memory.
- *A snapshot cannot say what it is missing.* Replaying a store five commits old found two files that had gained a reference since, and that appeared in no row. Checking the freshness of the files an answer names can never find them. Every query therefore asks git which files changed since each layer's snapshot (39 ms) and searches those for the name (3 ms), marking what it finds as heuristic text matches. An edit is never a silent miss.
- *The vendored upstream swamped the index.* With the Codex submodule included, 97% of the universal index of our repository was third-party code. A submodule is now indexed only when it is the user's own, by two offline tests (same host and owner as the superproject, and most recent commits by the superproject's authors), or when the user includes it; the choice is recorded in the agent's home, never in the repository, where a clone or the agent could write it. The URL alone does not tell a fork of someone else's project from one's own work: our Codex fork lives under our own namespace and is correctly left out.

Safety. Queries only read: they run inside the agent's sandbox, where `.git` is read-only. Indexing is a separate process the launcher starts outside it. Every index run of every kind is admitted against one host-wide memory budget and runs in a network-less Bubblewrap sandbox inside one systemd slice. In that sandbox a deliberately hostile build script could not write to the Cargo home, read a marker file in the home directory or overwrite the store, and the index was still produced. Starting the model server stops those scopes before its own pre-flight.

Results and gaps. On our repository, warm queries answer in 0.09 s at the median and 0.11–0.12 s at the 95th percentile, and `refs` on a TypeScript type in the upstream schema returns the same 48 files a whole-word search finds, in 40 ms. Since then the router has gained an `impact` query, a fallback that reports an unknown store schema and lets the other layer answer, and the submodule policy above; the Java and .NET indexers have run on small offline projects. Not built or not measured: search by meaning, `impact` used by the model, and the effect of an index run on decode speed. On SWE-bench the agent did not query the index unprompted (section 9).

11 Unattended overnight work

Tokens cost nothing on a machine one already owns, and it is idle at night. *Night Shift* lets the user queue coding tasks during the day (`/night add`, optionally with the test command that decides success) and finds a branch and a report for each in the morning. Its two halves share only files. The slash command, in the launcher crate behind one hook patch, writes a task record and never calls the model, so it answers at once, mid-turn included. The runner, in `ling-admin` and started by a systemd user timer, works through the queue.

Each task runs `ling exec` in a git worktree of its own, on a branch named after the task, inside a transient systemd scope with a memory cap. The runner, not the agent, commits: the agent's sandbox cannot write the repository's metadata. Nothing is merged, pushed or rebased. The live suite had shown a failure mode that matters when nobody is watching: the model sometimes announces a change and ends its turn (section 8). A task whose worktree is unchanged and whose last message announces work is therefore resumed with "go ahead", at most twice, and otherwise recorded as stalled. A task the window cuts off keeps its worktree and session and is resumed the next night.

The run never starts or loads the model server, and gives way to the user. Admission requires the server to be answering, the host-safety checks of section 7 to pass, memory headroom, no build or index run in progress, and a period in which no request reached the model. Two details came from measurement.

- *Parallelism cannot assume the full context.* The obvious rule, the KV pool divided by the context length, gives zero on the default model (section 6). Each task is budgeted 49,152 tokens, which gave two or three concurrent tasks depending on the launch. The budget can now be enforced as each task's compaction limit (`[night] compact_at`); it is off by default, so a task compacts at the launcher's 60% of the pool (section 6).
- *Idle is a counter, not a gauge.* A gauge of running requests sampled every 30 s misses short requests, so the run also requires the server's prompt-token counter to stay unchanged.

One task has run for real: a repository with a function that subtracts where it should add, and a failing test. Queued from a shell and listed in the interactive interface, it was fixed in 7 s; the detected test command passed; the commit sat on its own branch, and the checkout was untouched. That run exposed a defect: the commit also carried the byte-code files a test run leaves in a repository that does not ignore them. The runner now stages the agent's changes before its own test run, which removes its share of them; what the agent's own commands leave is still committed.

A day later an interrupted task was resumed on a second night and finished, and two tasks ran in parallel while a third waited at the memory gate, all live against the default model. A stall has still not been provoked live; it is covered by tests against a scripted stand-in for the agent. The runner's own test run, which executes code the model wrote, now runs in the agent's command sandbox with a policy the runner fixes: measured through the real path, it could write in its worktree and `/tmp` but not in the user's checkout, its `.git` or the home folder, and at `/airgapped on` it reached neither the internet nor the model server. Before the first task the run refreshes each repository's code index, and a task can be handed to a paired workstation's own runner (section 12).

12 Several workstations

A machine that serves the model is a *node*; anything else running `ling` is a *client*. A node uses its own loopback server and never looks for another. A client resolves the model server through fixed tiers: an explicit address, the node it remembered (found again by its identifier, never by address), and finally a multicast DNS browse; it never adopts a node other than the remembered one without asking. Every unattended caller of `ling` (Night Shift, the egress audit, SWE-bench) names its server explicitly, because a browse is a multicast DNS query and the audit would rightly count it as traffic.

Management goes over SSH, never an open port. Inference stays open on the local network, which we treat as trusted; control does not. `ling-admin node add` has the other node authorise a

key for one forced command, which refuses everything but its own operations: report, start, stop, set the served model, receive a model, unpair, and jobs. The other node's host key is pinned to its identifier at pairing, so a different machine answering at the same address later is refused. A *job* is a command sent to a node and run there as a systemd user unit with a mandatory memory cap and time limit, inside Bubblewrap with the home folder hidden and no GPU; its changes come back as a git branch. A paired node serving the same model is also an extra model server for Night Shift and SWE-bench, and a night task can be handed to its runner.

The second machine. On 2026-10-08 a second GB10 was installed from a release over SSH and paired with the first. The pairing itself worked as designed. It found two defects the tests had not: the browse had returned the node's link-local IPv6 address without its interface, which SSH cannot use (pairing now prefers IPv4 and adds the scope otherwise), and the installer, run with no terminal, had left the host settings and advertising as printed instructions rather than doing them (it now does every step, asking at most for one password at the start).

Addresses are not identities. The first night that used the second machine as an extra model server lost it: the machine's DHCP lease had moved, and every connection went to the address recorded at pairing. A node is now resolved on the network by its identifier before every connection; a recorded address is never connected to, and a node that does not answer a browse for a minute is refused rather than guessed. Resolving by browse then exposed the next layer. On the Wi-Fi chip of both machines, NetworkManager's default power saving lost up to about 80% of the multicast DNS queries sent to a node, so clients intermittently could not find it. The host check now reports power saving on a Wi-Fi interface, host setup and node preparation turn it off, and a client whose browse misses adds a one-line hint.

An engine for one model. The second machine is dedicated to a single-model inference engine for Qwen3.8-27B with DFlash2, written in C++ and CUDA and kept outside this paper's build. Its rule is exactness: every speed change must leave greedy output byte-identical to non-speculative decoding. After its batching stages it matched the production server on one stream (a step of about 103 ms) and came within 2% on two (33.4 against 33.9 tokens/s per stream), where the rest is steps that wait behind the other stream's prompt processing. A block of single-step savings followed, each on its own branch and kept only if bit-identical to the day's main and no slower at one stream and at two. Two 16-row tiles per block in the weight-streaming GEMM were kept: the hoped-for 6 ms on one stream was not there, since the step's GEMMs already streamed at 233–244 GB/s at 1–16 rows, but at 17–32 rows, the batched verify of two sequences, the narrow matrices had fallen to 110–190 GB/s; the change took the per-stream step period at two streams from 154.2 to 147.5 ms (32.6 to 34.1 tokens/s per stream). The commit's state replay on a second CUDA stream beside the next draft was kept: a step of 102.6 to 101.4 ms on one stream (48.6 to 49.2 tokens/s), a per-stream period of 147.8 to 146.3 ms on two (34.0 to 34.3). A third `cp.async` stage in the verify attention was rejected: the kernel already read its keys at the plain-read rate, and the stage made the batched step in place slower (122.3 to 124.1 ms). Those comparisons are at equal acceptance, and a methods fix made them so: the replay had sent no sampling seed, so each request drew its own acceptance and two runs were two workloads, which puts noise of a few percent on every earlier figure here, 33.4 against 33.9 included. A comparison is now reported only when both runs produce the same tokens in the same number of steps. Decode on this machine is bound by reading the weights, 17.6 GB per step, so the remaining gains are in fewer bytes read per accepted token. We leave the engine to future work.

13 Lessons

Keep the fork boring. Every behaviour that could live in a linked crate went there. Every remaining hook is a single call or predicate. The measure of success is how long an upstream bump takes, not how little code we wrote. A 43 KB series of one- and two-line hooks across 43 files turns an upstream release into an afternoon's review rather than a merge project.

Presentation diffs are the expensive ones. Almost all of the original series' size, 97%, was a one-word rename inside machine-formatted data. Renaming at run time, or at the point where presentation is assembled, removed it. We expect this to be the common case for rebranded forks.

Local models change the tool contract. A harness tuned for a frontier hosted model exposes tools in ways a smaller local model may not use, and in ways a local server may silently drop: twice, a whole class of tools never reached the model. It is cheaper to adapt the tool surface to the model and its server, here by shell commands and by translating tool types at the boundary, than to wait for either to adapt to the surface.

Measure the noise before the effect. The same configuration, run three times on the same 44 tasks, resolved between 29 and 31: 23 tasks every time, 8 never, and the other 13 in some runs but not in others. Every candidate we have tested so far moved the count by three tasks or fewer. A single run of each arm can only reject large effects, so each candidate is first judged by the behaviour it was meant to change. The same lesson reached the two measuring instruments: the engine's replays carried sampled-acceptance noise until they were seeded, and a benchmark night whose candidate arm died at start in every task still exited 0 until each arm was made to run one task to its end before the night begins (section 9).

Unified memory needs admission control, not just limits. A container memory cap bounds the container. It does not decide who dies when the host runs short, and it does not prevent a reclaim livelock in which nothing dies. Pre-flight checks, pressure-based watchdogs and caps on every auxiliary job were all necessary.

Parallel work needs discipline, even in one working tree. During development, several builds and tasks often ran concurrently in one working tree. Three failure modes appeared that are familiar from team development, in faster form.

- Concurrent builds wiped each other's shared source tree. A file lock in the builder fixed this.
- A copy that preserved file timestamps let Cargo link a stale launcher. Copying with fresh timestamps fixed this.
- Commits swept up another task's staged changes, in one case deleting the patch series from a commit. Checking the index before every commit and staging only named hunks prevented recurrences.

Separately, a stopped release task had already triggered a workflow that published a release without the agent binaries. It was cancelled too late and then deleted. Outward-facing actions started by automation need their own kill switch.

14 Related work

SWE-agent [23] and OpenHands [22] study the agent-computer interface and evaluate on SWE-bench [12]. OpenHands is also developed as a platform for everyday use. Our contribution is complementary. We do not propose a new agent; we show how to take a vendor's production client, keep it upgradable, and make it vendor-free and self-contained. Code-navigation work for agents, including LocAgent's code graphs [5], CodeNib's multi-view context serving [25] and tree-sitter knowledge graphs served over MCP [21], informs section 10. On the serving side we rely on vLLM's paged attention [13], speculative decoding [14] and block-diffusion drafting [4]. Our contribution there is the host-level safety that unified memory requires, rather than any serving algorithm.

15 Limitations

- **Task-level accuracy is an estimate from a small sample.** The SWE-bench figures (section 9) come from 24 and 100 tasks, run locally on community images, one session per task, and are mapped to an overall score by models fitted on published submissions. They are not leaderboard results, and run-to-run variation on the same tasks is several points.
- **One kind of machine, one model.** All results come from GB10 workstations with one model configuration.
- **Agent-session throughput is a replay.** The figures for agent sessions come from replaying recorded sessions against the production configuration on the second machine; the replayed outputs reach 77% of the recorded lengths, and they were taken before the replay sent a sampling seed (section 12), so they carry acceptance noise of a few percent.
- **Upstream's own tests are not green.** In the last full run of Codex's own suite against the patched tree (20,380 tests, 20,362 passed), sixteen app-server tests timed out for a reason we have not confirmed. That run predates the patches from 0017 on, which change the screen snapshots that list the slash commands; the suite has not been re-run since.
- **The live suite predates the current series.** The slash-command suite of table 3 was last run on the 16-patch build.
- **The code index is partial.** C and C++ have no exact layer on this architecture, the Java and .NET indexers have run only on small projects, and neither search by meaning nor the effect of an index run on decode speed has been measured; on SWE-bench the agent did not use the index unprompted (section 10).
- **Night Shift has run a handful of tasks.** A stall has not been provoked live (section 11).
- **The advertised context still exceeds the KV pool.** The compaction limit keeps a session below the pool, but what the server does with a larger prompt is untested (section 6).
- **The messengers are lightly tested.** The Signal bridge has run live against one account; the Matrix and Telegram bridges only against stand-ins.

16 Conclusion

Mightling shows that a production coding-agent client can be made vendor-free, with its model on a single unified-memory workstation, without giving up the ability to take upstream releases.

The localisation took about 9 KB of patches; the series is now 43.0 KB because every later feature (a terse-answer mode, an overnight queue, an enforced network switch, flattened MCP and Code Mode tools, connectors without a vendor sign-in, workstation pairing) cost a hook in Codex and put its logic in crates of ours, with no change to the method. The cloud surface is audited and neutralised explicitly and re-checked by a trace after every build, and the host is protected by admission checks and a pressure watchdog. On a 100-task sample of SWE-bench Verified the agent's local result corresponds to about 69% overall; neither a study-then-fix mode, nor four prompt and harness changes, nor Code Mode's `exec` measured above the run-to-run noise. The next steps are compaction that drops old tool output before it summarises, choosing among several attempts where time allows (overnight), and speed: an engine for this one model, on the second workstation, already matches the production server, is taking milliseconds off each step with its output bit-identical, and has to pass production on seeded replays of both.

Availability

The system is developed in a public repository (`github.com/dreamference/mightling`). The patch series, launcher crates, builder and specifications described here are licensed AGPL-3.0-or-later. Codex is used under its upstream Apache-2.0 licence, which permits combination with AGPL-licensed code; its notices are retained. The Codex submodule tracks a public fork (`github.com/dgxcoder/codex`), which is an unmodified copy of upstream at the pinned tag.

Measurements

All numbers in this paper were measured on the author's machines during development or read from the repository, and each arXiv citation was checked against the arXiv record.

References

[1] Morayo Danielle Adeyemi, Ryan A. Rossi, and Franck Dernoncourt. CAVEWOMAN: How large language models behave under linguistic input and output compression. *arXiv preprint arXiv:2606.24083*, 2026.

[2] Anthropic. Model context protocol specification. `https://modelcontextprotocol.io`, 2025.

[3] Max Brunsfeld and tree-sitter contributors. Tree-sitter: an incremental parsing system for programming tools. `https://github.com/tree-sitter/tree-sitter`, 2026.

[4] Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.

[5] Zhaoling Chen, Xiangru Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. *arXiv preprint arXiv:2503.09089*, 2025.

[6] Wenhua Cheng, Weiwei Zhang, Haihao Shen, Yiyang Cai, Xin He, Kaokao Lv, and Yi Liu. Optimize weight rounding via signed gradient descent for the quantization of LLMs. *arXiv preprint arXiv:2309.05516*, 2023.

[7] Codex contributors. Codex CLI: a lightweight coding agent that runs in your terminal. https://github.com/openai/codex, 2026. Release rust-v0.158.0, commit 064c6b8.

[8] Deno authors. rusty_v8: Rust bindings for the V8 JavaScript engine. https://github.com/denoland/rusty_v8, 2026.

[9] DeusData. codebase-memory-mcp. https://github.com/DeusData/codebase-memory-mcp, 2026. Release v0.11.0, MIT.

[10] greynewell. SWE-bench Verified images for arm64. Docker Hub, greynewell/swe-bench-arm64, 2026. Community build; 400 of the 500 Verified instances.

[11] Intel. Qwen3.5-122B-A10B int4 AutoRound checkpoint. https://huggingface.co/Intel/Qwen3.5-122B-A10B-int4-AutoRound, 2026.

[12] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.

[14] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.17192.

[15] Onyx contributors. Onyx: open-source AI chat and enterprise search. https://github.com/onyx-dot-app/onyx, 2026.

[16] SCIP contributors. SCIP code intelligence protocol. https://github.com/scip-code/scip, 2026. Apache-2.0.

[17] SearXNG contributors. SearXNG: a privacy-respecting metasearch engine. https://github.com/searxng/searxng, 2026.

[18] Denis Shiryaev. Does speaking to agents like cavemen really save 65% of tokens? We test. JetBrains AI Blog, https://blog.jetbrains.com/ai/2026/07/speak-to-ai-agents-like-cavemen-tosave-tokens/, 2026. Accessed 2026-10-01.

[19] SWE-bench team. SWE-bench experiments: published submissions and per-instance results. https://github.com/SWE-bench/experiments, 2026. Commit 40f164d5, 2026-09-03.

[20] The Linux Kernel documentation. PSI – pressure stall information. https://docs.kernel.org/accounting/psi.html.

[21] Martin Vogel, Falk Meyer-Eschenbach, Severin Kohler, Elias Grünewald, and Felix Balzer. Codebase-memory: Tree-sitter-based knowledge graphs for LLM code exploration via MCP. *arXiv preprint arXiv:2603.27277*, 2026.

[22] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[23] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.

[24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

[25] Zhongming Yu, Hengjia Yu, Boqin Yuan, Shuting Zhao, Yizhao Chen, Aryan Dokania, Mihir Jagtap, Jiayu Chang, Yitong Ma, Yash Jayswal, Wentao Ni, Hejia Zhang, Zhaoling Chen, Gangda Deng, and Jishen Zhao. CodeNib: A multi-view data system for serving repository context to coding agents. *arXiv preprint arXiv:2607.25431*, 2026.

[26] z-lab. Qwen3.5-122B-A10B DFlash drafter. `https://huggingface.co/z-lab/Qwen3.5-122B-A10B-DFlash`, 2026.

[27] z-lab and maurienne-ai. Qwen3.8-27B DFlash2 drafter, NVFP4 build. `https://huggingface.co/maurienne-ai/Qwen3.8-27B-DFlash2-NVFP4-RTNcal`, 2026. An NVFP4 build of `https://huggingface.co/z-lab/Qwen3.8-27B-DFlash2`.